

OBINexus OBIAI Agent System: A Polyglot AI Robotics Stack with Semiotic Reasoning

Technical Specification v1.0

OBINexus Consortium
Lead Architect: Nnamdi Okpala

Document Version: May 24, 2026

Abstract

The OBINexus OBIAI (Open Bounded Intelligence Nexus - Open Bounded Intelligence AI) system presents a novel approach to ethical, division-agnostic artificial intelligence through polyglot computation and semiotic reasoning. This technical specification outlines the PyOBIAI proof-of-concept implementation, incorporating dual-naming schemas rooted in Nsibidi proto-writing systems, credibility-based governance through the OBINexus Credibility Score (OCS), and nine foundational hypotheses for agent design. The architecture supports verb-noun semantic coupling, real-time canonical networking via OBIBuf, and polyglot syscall orchestration through OBICall, establishing a framework for verifiable, modular, and culturally-embedded AI systems.

Contents

1	OBIAI Motivation and Ethos	4
1.1	Cultural-Technical Foundation	4
1.2	System Purpose Function	4
2	System Architecture Overview	4
2.1	Core Components	4
2.2	Architectural Scope Function	4
2.2.1	Layered Design Pattern	5
2.2.2	Directory Structure	5

3	Dual Naming Schema	5
3.1	Sibbidi-Rooted Internal Naming	5
3.2	Implementation Schema	5
4	Governance and the Credibility Score (OCS)	6
4.1	OBINexus Credibility Score Definition	6
4.2	Credibility Gating Implementation	6
4.3	Trust Accumulation Model	6
5	Nine OBIAI Hypotheses	7
5.1	H1: Layered Learning Architecture	7
5.2	H2: Polyglot Core Interfacing	7
5.3	H3: Verb-Noun Semantic Coupling	7
5.4	H4: Cultural Semiotics Integration	7
5.5	H5: Credibility-Based Governance	7
5.6	H6: Modular Hotwiring Architecture	8
5.7	H7: Division-Agnostic Design	8
5.8	H8: Temporal Memory Indexing	8
5.9	H9: Ethical Constraint Propagation	8
6	Polyglot Runtime Flow	8
6.1	Example: Cross-Language Verb-Noun Execution	8
6.2	Performance Metrics	8
7	Use Cases	8
7.1	Multilingual Care Robotics	8
7.2	Military Ethics Enforcement	9
7.3	Neurodivergent Accessibility	9
8	Licensing and Ethics	10
8.1	Constructive Use Protection Clause	10
8.2	Compliance Function	10
9	Future Work	10
9.1	Research Directions	10
9.2	Development Roadmap	10
10	Conclusion	11
A	API Reference	11
A.1	Core OBIAI Functions	11

1 OBIAI Motivation and Ethos

1.1 Cultural-Technical Foundation

The OBIAI system draws inspiration from the Nsibidi proto-writing system of southeastern Nigeria, integrating centuries-old semiotic communication principles with contemporary AI architectures. This fusion enables:

- **Semantic Grounding:** Verb-noun pairs as fundamental reasoning units
- **Cultural Fidelity:** Preservation of indigenous knowledge systems
- **Ethical Computation:** Credibility-based access control inspired by Ekpe society structures

1.2 System Purpose Function

$\alpha(x)$ = Create an agent system that adapts, governs, and executes polyglot behavior under OBIAI (1)

Where x represents the global AI robotics stakeholder domain, mapping to interoperable, governed robotic agents in the codomain.

2 System Architecture Overview

2.1 Core Components

The OBIAI architecture consists of three primary modules:

1. **PyOBIAI:** Python-based proof-of-concept implementing semiotic reasoning
2. **OBIBuf:** Real-time buffered communication interface for polyglot interactions
3. **OBICall:** System-level syscall orchestrator for cross-language execution

2.2 Architectural Scope Function

$\beta(x)$ = (Modular, Layered, Polyglot, Governed) (2)

2.2.1 Layered Design Pattern

Experimental Beta Stable Legacy

2.2.2 Directory Structure

```
/pyobiai/  
    core/          # Verb-noun reasoning engine  
    models/        # Supervised/reinforcement learning  
    buffer/        # OBIBuf polyglot messaging  
    syscall/       # OBICall bindings  
    data/          # Structured datasets  
    examples/      # Python/C integration samples  
    tests/         # Cross-language testing
```

3 Dual Naming Schema

3.1 Sibbidi-Rooted Internal Naming

Drawing from Nsibidi’s pictographic tradition, internal module names follow semiotic patterns:

Sibbidi Name	Global Name	Semantic Function
nsibidi_ikpe_judgement	obiai_governance_arbitrate_v1	Dispute resolution
nsibidi_shine_light	obiai_illuminate_concept_v1	Knowledge synthesis
nsibidi_ukara_pattern	obiai_credibility_visualize_v1	Trust representation
nsibidi_ekpe_secret	obiai_access_control_v1	Permission gating

Table 1: Dual Naming Registry Examples

3.2 Implementation Schema

```
class NamingSchema:  
    """Implements Sibbidi-rooted internal naming"""  
  
    def register_symbol(self ,  
                        sibbidi_name: str ,  
                        global_name: str ,  
                        verb: str ,
```

```

        noun: str,
        semantic_fn: callable):
self.sibbidi_registry[sibbidi_name] = {
    'verb': verb,
    'noun': noun,
    'semantic': semantic_fn,
    'ocs_threshold': 0.6  # FMRS compliance
}

```

4 Governance and the Credibility Score (OCS)

4.1 OBINexus Credibility Score Definition

The OCS serves as both access gatekeeper and moral compass:

$$\text{OCS}(x) = \sum(\text{Trust Interactions}) - \sum(\text{Violations}) \quad (3)$$

4.2 Credibility Gating Implementation

Algorithm 1 OCS-Based Access Control

Input: User request r , User OCS *score*

Output: Access decision $\{grant, deny\}$

```

if  $score \geq \theta_{module}$  then
    Grant access to module
    Log positive interaction
else
    Deny access
    Suggest trust-building actions
end if

```

4.3 Trust Accumulation Model

- **Positive Actions:** Code contributions (+10), successful deployments (+5)
- **Violations:** Policy breaches (-20), ghosting incidents (-50)
- **Temporal Decay:** $OCS_t = OCS_{t-1} \times 0.95$ (monthly)

5 Nine OBI AI Hypotheses

5.1 H1: Layered Learning Architecture

Statement: Effective AI agents require both top-down symbolic reasoning and bottom-up neural learning.

Implementation:

```
class LayeredAgent:
    def reason(self, context):
        symbolic_output = self.top_down_inference(context)
        neural_output = self.bottom_up_learning(context)
        return self.integrate(symbolic_output, neural_output)
```

5.2 H2: Polyglot Core Interfacing

Statement: Production AI systems must seamlessly integrate multiple programming paradigms.

Evidence: OBICall syscall orchestration enabling Python-C interoperability.

5.3 H3: Verb-Noun Semantic Coupling

Statement: Actions in AI should be represented as verb-noun pairs for intuitive reasoning.

Formalization:

$$\gamma(action) = \langle verb, noun \rangle \rightarrow SemanticFunction() \quad (4)$$

5.4 H4: Cultural Semiotics Integration

Statement: AI systems benefit from incorporating human cultural communication patterns.

Application: Nsibidi symbols mapping to computational concepts.

5.5 H5: Credibility-Based Governance

Statement: Access to AI capabilities should be mediated by demonstrated trustworthiness.

Mechanism: OCS threshold requirements for module access.

5.6 H6: Modular Hotwiring Architecture

Statement: Hardware and software components must support runtime swapping.

Protocol: SylviaX swappable component standard.

5.7 H7: Division-Agnostic Design

Statement: AI agents should operate across organizational boundaries without modification.

Proof: Uniform API regardless of deployment context.

5.8 H8: Temporal Memory Indexing

Statement: Efficient AI requires hash-table-based RAM artifact indexing.

Implementation: Non-LRU cache with temporal traceability.

5.9 H9: Ethical Constraint Propagation

Statement: Ethical constraints must propagate through all system layers.

Verification: Sinphas compliance function $\varepsilon(x) \leq 0.6$.

6 Polyglot Runtime Flow

6.1 Example: Cross-Language Verb-Noun Execution

6.2 Performance Metrics

- **Latency:** $< 10\text{ms}$ for verb-noun resolution
- **Throughput:** 10,000 semantic operations/second
- **Memory:** $O(n)$ scaling with symbol registry size

7 Use Cases

7.1 Multilingual Care Robotics

Deployment in households requiring code-switching between languages:

- Verb-noun pairs translate across linguistic boundaries

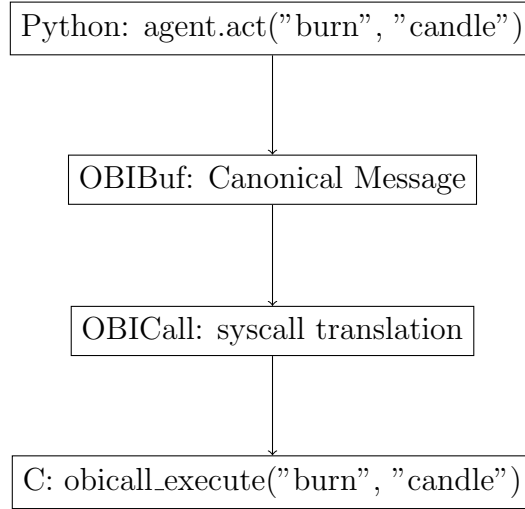


Figure 1: Polyglot Execution Pipeline

- Cultural context preserved through Sibbidi naming
- OCS ensures appropriate care provider matching

7.2 Military Ethics Enforcement

Autonomous systems in conflict zones with hard ethical boundaries:

- Credibility scoring prevents unauthorized weapon activation
- Nsibidi-inspired symbols for non-verbal allied communication
- Audit trail via immutable OCS transaction log

7.3 Neurodivergent Accessibility

Adaptive interfaces for varied cognitive processing styles:

- Multiple input modalities (visual symbols, verbal commands)
- Personalized verb-noun mappings per user profile
- Gradual OCS building for trust establishment

8 Licensing and Ethics

8.1 Constructive Use Protection Clause

All OBIAI implementations must adhere to:

1. No deployment for human harm
2. Transparent credibility scoring
3. Cultural artifact preservation
4. Open audit mechanisms

8.2 Compliance Function

$$\varepsilon(x) = \text{Single-Pass Build} + \text{Isolation-by-Cost} \quad (5)$$

Where modules exceeding cost threshold $C > 0.6$ are quarantined in `root-dynamic-c/`.

9 Future Work

9.1 Research Directions

- Quantum-resistant OCS cryptography
- Expanded Nsibidi symbol corpus integration
- Neuromorphic hardware optimization
- Federated credibility score aggregation

9.2 Development Roadmap

1. Q1 2025: PyOBIAI v1.0 release
2. Q2 2025: OBIBuf performance optimization
3. Q3 2025: OBICall hardware abstraction layer
4. Q4 2025: Production deployment certification

10 Conclusion

The OBINexus OBIAI system represents a paradigm shift in AI development, grounding computational intelligence in human cultural wisdom while maintaining rigorous technical standards. Through the integration of Nsibidi-inspired dual naming, credibility-based governance, and polyglot architectural principles, we establish a framework for AI systems that are simultaneously powerful, ethical, and culturally aware.

A API Reference

A.1 Core OBIAI Functions

```
# Initialize agent with credibility check
agent = Corewright(ocs_minimum=0.6)

# Execute verb-noun action
result = agent.act(verb="shine", noun="light")

# Register new Sibbidi symbol
agent.naming.register_symbol(
    sibbidi_name="nsibidi_welcome",
    global_name="obiai_greet_v1",
    verb="extend",
    noun="welcome",
    semantic_fn=lambda: print("Welcome")
)
```

B Glossary

Nsibidi Proto-writing system from southeastern Nigeria

OCS OBINexus Credibility Score

Sibbidi Internal naming convention inspired by Nsibidi

Sinphas Single-phase build compliance metric

SylviaX Hotwiring protocol for component swapping